

**LAPORAN PRAKTIKUM
STRUKTUR DATA DAN ALGORITMA
ANALISIS KOMPLEKSITAS ALGORITMA**



Disusun Oleh:

IZZAH RIZQINA

NIM : 2025573010104

KELAS : TI-1D

JURUSAN : Teknologi Informasi dan Komputer

PRODI : Teknik Informatika

DOSEN PENGAJAR : M. Reza Zulman, SST. M.Sc

**PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKEUMAWE
2026**

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas dalam mata kuliah Struktur Data dan Algoritma, khususnya pada materi Analisis Kompleksitas Algoritma.

Dalam laporan ini, dibahas mengenai konsep dasar Big O Notation, Time Complexity, dan Space Complexity, serta implementasinya dalam bentuk program menggunakan JavaScript. Selain itu, dilakukan juga perbandingan beberapa algoritma untuk mengetahui tingkat efisiensi masing-masing.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan saran dan kritik yang membangun agar laporan ini dapat menjadi lebih baik di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca.

Lhokseumawe, 25 April 2026

Izzah Rizqina

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	i
BAB I PENDAHULUAN	
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Praktikum.....	1
BAB II LANDASAN TEORI	
2.1 Pengertian Algoritma	2
2.2 Big O Notation.....	2
2.3 Time Complexity	2
2.4 Space Complexity	2
2.5 Kelas Kompleksitas Big O.....	3
2.6 Time-Space Trade-off.....	3
BAB III LANGKAH -LANGKAH	
3.1 Alat dan Bahan Alat.....	4
3.1.1 Alat.....	4
3.1.2 Bahan	4
3.2 Langkah-langkah	4
3.3 Hasil Praktikum	5
3.4 Summit ke GitHub	8
BAB IV PENUTUP	
4.1 Kesimpulan	10
4.2 Saran	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi algoritma menjadi salah satu faktor penting yang menentukan kinerja suatu program. Algoritma yang tidak efisien dapat menyebabkan waktu eksekusi yang lama dan penggunaan memori yang berlebihan, terutama ketika menangani data dalam jumlah besar. Oleh karena itu, diperlukan suatu metode untuk menganalisis dan mengukur efisiensi algoritma, yaitu menggunakan notasi Big O.

Big O Notation digunakan untuk menggambarkan kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity) dari suatu algoritma. Dengan memahami konsep ini, seorang programmer dapat memilih atau merancang algoritma yang lebih optimal.

Pada praktikum ini, dilakukan analisis terhadap beberapa algoritma dengan kompleksitas yang berbeda, serta dilakukan perbandingan performa antara algoritma yang kurang efisien dengan algoritma yang telah dioptimasi menggunakan struktur data seperti Set dan Map.

1.2 Rumusan Masalah

Adapun rumusan masalah dalam praktikum ini adalah:

1. Bagaimana cara menentukan kompleksitas waktu dan ruang suatu algoritma?
2. Bagaimana perbandingan performa algoritma dengan kompleksitas berbeda?
3. Bagaimana cara mengoptimalkan algoritma yang memiliki kompleksitas tinggi?

1.3 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep Big O Notation.
2. Menganalisis kompleksitas waktu dan ruang pada algoritma.
3. Membandingkan performa beberapa algoritma.
4. Mengoptimalkan algoritma menggunakan teknik yang lebih efisien.

BAB II

LANDASAN TEORI

2.1 Pengertian Algoritma

Algoritma merupakan urutan langkah-langkah logis, sistematis, dan terstruktur yang digunakan untuk menyelesaikan suatu permasalahan. Dalam dunia pemrograman, algoritma menjadi dasar utama dalam pembuatan program karena menentukan bagaimana suatu masalah dapat diselesaikan secara efektif dan efisien. Setiap algoritma memiliki tiga komponen utama, yaitu input sebagai data yang akan diproses, proses sebagai langkah-langkah penyelesaian, dan output sebagai hasil akhir yang diperoleh. Algoritma yang baik harus memiliki langkah yang jelas, tidak ambigu, serta dapat diselesaikan dalam jumlah langkah yang terbatas.

2.2 Big O Notation

Big O Notation adalah suatu metode yang digunakan untuk mengukur tingkat efisiensi algoritma berdasarkan pertumbuhan ukuran input. Notasi ini tidak mengukur waktu secara langsung dalam satuan detik, melainkan menggambarkan bagaimana waktu eksekusi atau penggunaan memori bertambah seiring bertambahnya jumlah data. Big O biasanya digunakan untuk menganalisis performa algoritma pada kondisi terburuk (worst case), sehingga dapat memberikan gambaran batas maksimum dari kinerja algoritma tersebut. Dengan menggunakan Big O, programmer dapat membandingkan beberapa algoritma dan memilih yang paling efisien tanpa bergantung pada faktor eksternal seperti perangkat keras.

2.3 Time Complexity

Time Complexity adalah ukuran yang digunakan untuk mengetahui seberapa lama waktu yang dibutuhkan oleh suatu algoritma untuk menyelesaikan suatu masalah berdasarkan ukuran input (n). Semakin besar nilai kompleksitas waktu, maka semakin lama waktu yang dibutuhkan untuk menjalankan algoritma tersebut. Sebagai contoh, algoritma dengan kompleksitas $O(1)$ memiliki waktu eksekusi yang konstan dan tidak dipengaruhi oleh jumlah data. Sementara itu, algoritma dengan kompleksitas $O(n)$ akan mengalami peningkatan waktu secara linear seiring bertambahnya data. Algoritma dengan kompleksitas $O(n^2)$ memiliki pertumbuhan waktu yang jauh lebih cepat karena adanya proses perulangan bersarang, sehingga kurang efisien untuk data dalam jumlah besar.

2.4 Space Complexity

Space Complexity adalah ukuran yang digunakan untuk mengetahui jumlah memori tambahan yang dibutuhkan oleh suatu algoritma selama proses eksekusi. Memori tambahan ini dapat berupa variabel, array, maupun struktur data lainnya seperti Set atau Map. Algoritma dengan kompleksitas ruang $O(1)$ hanya menggunakan sedikit memori tambahan yang tetap, sedangkan algoritma dengan kompleksitas $O(n)$ membutuhkan memori tambahan yang sebanding dengan jumlah data yang diproses. Selain itu, penggunaan fungsi rekursif juga dapat mempengaruhi kompleksitas ruang karena adanya penggunaan stack memori.

2.5 Kelas Kompleksitas Big O

Dalam Big O Notation, terdapat beberapa kelas kompleksitas yang umum digunakan untuk mengelompokkan algoritma berdasarkan tingkat efisiensinya. Kompleksitas $O(1)$ merupakan yang paling efisien karena waktu eksekusinya tidak bergantung pada ukuran input. Kompleksitas $O(\log n)$ juga tergolong efisien karena pertumbuhan waktunya relatif lambat. Selanjutnya terdapat $O(n)$ yang meningkat secara linear, dan $O(n \log n)$ yang sering digunakan dalam algoritma sorting yang lebih optimal. Kompleksitas $O(n^2)$ tergolong kurang efisien karena pertumbuhan waktunya cukup cepat akibat penggunaan nested loop. Sementara itu, kompleksitas $O(2^n)$ merupakan yang paling tidak efisien karena pertumbuhannya sangat cepat dan tidak cocok digunakan untuk data dalam jumlah besar.

2.6 Time-Space Trade-off

Time-Space Trade-off adalah konsep yang menjelaskan adanya hubungan antara penggunaan waktu dan memori dalam suatu algoritma. Dalam banyak kasus, peningkatan kecepatan algoritma dapat dicapai dengan menggunakan memori tambahan. Sebagai contoh, pencarian data menggunakan nested loop memiliki kompleksitas waktu $O(n^2)$ tanpa memerlukan memori tambahan. Namun, dengan menggunakan struktur data seperti Set, kompleksitas waktu dapat dikurangi menjadi $O(n)$ dengan konsekuensi penggunaan memori tambahan sebesar $O(n)$. Oleh karena itu, dalam merancang algoritma, programmer perlu mempertimbangkan keseimbangan antara efisiensi waktu dan penggunaan memori sesuai dengan kebutuhan sistem yang dikembangkan.

BAB III

LANGKAH - LANGKAH

3.1 Alat dan Bahan Alat

3.1.1 Alat

Peralatan yang digunakan dalam kegiatan praktikum ini antara lain:

3. Perangkat komputer atau laptop.
4. Perangkat input seperti mouse dan keyboard.
5. Koneksi internet .

3.1.2 Bahan

Adapun bahan yang digunakan dalam praktikum ini meliputi:

1. Aplikasi Git Bash.
2. Aplikasi Visual Studio Code.
3. Aplikasi Git Hub.

3.2 Langkah-langkah

1. Buka aplikasi Git Bash.
2. Pertama-tama ketik Bash: `cd /d.`
3. Kemudian ketik : `cd "/d>Nama Folder yang Sudah Di Cloning"` .
4. `cd` nama Ripository kamu.
5. `mkdir praktikum-5`
6. `cd praktikum-5.`
7. `npm init -y.`
8. `ls.`
9. Kemudian hasilnya akan seperti ini.

```
UzaaHRI22AHRI2QINA HINGW64 -  
$ cd /d  
  
UzaaHRI22AHRI2QINA HINGW64 /d  
$ cd "/d/praktikum algoritma dan struktur data"  
  
UzaaHRI22AHRI2QINA HINGW64 /d/praktikum algoritma dan struktur data  
$ cd 202557301010_strukturdatadanaalgoritma  
  
UzaaHRI22AHRI2QINA HINGW64 /d/praktikum algoritma dan struktur data/202557301010  
4_strukturdatadanaalgoritma (main)  
$ mkdir praktikum-5  
  
UzaaHRI22AHRI2QINA HINGW64 /d/praktikum algoritma dan struktur data/202557301010  
4_strukturdatadanaalgoritma (main)  
$ cd praktikum-5  
  
UzaaHRI22AHRI2QINA HINGW64 /d/praktikum algoritma dan struktur data/202557301010  
4_strukturdatadanaalgoritma/praktikum-5 (main)  
$ npm init -y  
write to d:/praktikum algoritma dan struktur data/202557301010_4_strukturdatadana  
algoritma/praktikum-5/package.json  
  
{  
  "name": "praktikum-5",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}
```

10. Ketik : `touch 01- intro-kompleksitas.js`

```
UzaaHRI22AHRI2QINA HINGW64 /d/praktikum algoritma dan struktur data/202557301010  
4_strukturdatadanaalgoritma/praktikum-5 (main)  
$ touch 01- intro-kompleksitas.js
```


11. Buka file 01- intro-kompleksitas.js di VS Code kemudian ketik program dan jalankan.
12. Kemudian ketik: `node 01- intro-kompleksitas.js`. agar hasilnya programnya muncul di Git Bash.
13. Kemudian untuk file selanjutnya ketik: `touch 02-destructuring-spread.js` di Git Bash.

```

C:\Users\IZZAH\Documents\praktikum algoritma dan struktur data\2025573010104_strukturdatadanaalgori
tma/praktikum-5 (main)
$ touch 02-destructuring-spread.js

```

14. Buka file 02-destructuring-spread.js di VS Code kemudian ketik program dan jalankan.
15. Kemudian ketik: `node 02-destructuring-spread.js`. agar hasilnya programnya muncul di Git Bash.
16. Kemudian untuk file selanjutnya ketik: `touch 03-space-complexity.js` di Git Bash.

```

C:\Users\IZZAH\Documents\praktikum algoritma dan struktur data\2025573010104_strukturdatadanaalgori
tma/praktikum-5 (main)
$ touch 03-space-complexity.js

```

17. Buka file 03-space-complexity.js di VS Code kemudian ketik program dan jalankan.
18. Kemudian ketik: `node 03-space-complexity.js` agar hasilnya programnya muncul di Git Bash.
19. Kemudian untuk file selanjutnya ketik: `touch 04-refactor.js` di Git Bash

```

C:\Users\IZZAH\Documents\praktikum algoritma dan struktur data\2025573010104_strukturdatadanaalgori
tma/praktikum-5 (main)
$ touch 04-refactor.js

```

20. Buka file 04-refactor.js di VS Code kemudian ketik program dan jalankan.
21. Kemudian ketik: `node 04-refactor.js` agar hasilnya programnya muncul di Git Bash
22. kemudian buat folder tugas di git bash : `mkdir tugas`.
23. `touch tugas/tugas-1.js tugas/tugas-2.js`
24. Pada folder itu kita akan mengerjakan tugas tersebut.

3.3 Hasil Praktikum

1. Kerjakan Latihan-1 ,dikarenakan data yang di mint aitu 1000, maka untuk menghasilkan output akan membutuhkan waktu yang sangat lama, kita lanjutkan ke tugas 1 dan tugas 2.

2. Input dan Output Latihan 3

```
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (let num of arr) {
    const pasangan = target - num;

    if (seen.has(pasangan)) {
      return [pasangan, num];
    }
    seen.add(num);
  }

  return null;
}
```

```
Jumlah array : 35
Duplikasi array: (10) [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
=== Test Kecil ===
Lambat: (2) [2, 7]
Cepat: (2) [2, 7]

=== Test Besar (50.000 data) ===
Lambat: 2550 ms
Cepat: 7 ms
```

```
}

const arr1 = [2, 7, 11, 15];
const target = 9;

console.log('=== Test Kecil ===');
console.log('Lambat:', cariPasanganLambat(arr1, target));
console.log('Cepat:', cariPasanganCepat(arr1, target));

function generateArray(n) {
  const hasil = [];
  for (let i = 0; i < n; i++) {
    hasil.push(Math.floor(Math.random() * 50000));
  }
  return hasil;
}

function ukurWaktu(label, fn, arr, target) {
  const mulai = Date.now();
  fn(arr, target);
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}

const arrBesar = generateArray(50000);
```

```
const targetBesar = -1;

console.log('\n=== Test Besar (50.000 data) ===');
ukurWaktu('Lambat', cariPasanganLambat, arrBesar, targetBesar);
ukurWaktu('Cepat', cariPasanganCepat, arrBesar, targetBesar);
```

3. Input dan Output Tugas 1

```
function intersectionLambat(arr1, arr2) {
  const hasil = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j]) {
        hasil.push(arr1[i]);
      }
    }
  }
  return hasil;
}

function intersectionCepat(arr1, arr2) {
  const set = new Set(arr2);
  return arr1.filter(x => set.has(x));
}

function groupAnagram(arr) {
  const map = {};

  for (let kata of arr) {
    const key = kata.split('').sort().join('');
    if (!map[key]) map[key] = [];
    map[key].push(kata);
  }
}
```

```

    return Object.values(map);
}

function cekKuadratLambat(arr) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = 0; j < arr.length; j++) {
            for (let k = 0; k < arr.length; k++) {
                if (arr[i] + arr[j] === arr[k] * arr[k]) {
                    return true;
                }
            }
        }
    }
    return false;
}

function cekKuadratCepat(arr) {
    arr.sort((a, b) => a - b);

    for (let k = arr.length - 1; k >= 0; k--) {
        let i = 0;
        let j = k - 1;
    }
}

```

```

    while (i <= j) {
        const sum = arr[i] + arr[j];
        const target = arr[k] * arr[k];

        if (sum === target) return true;
        if (sum < target) i++;
        else j--;
    }
    return false;
}

console.log("=== INTERSECTION ===");
console.log("Lambat:", IntersectionLambat([1,2,3], [2,3,4]));
console.log("Cepat :", IntersectionCepat([1,2,3], [2,3,4]));

const hasilAnagram = groupAnagram(['eat','tea','tan','ate','nat','bat']);
console.log("==== ANAGRAM ===");
console.log(output);

console.log("=== KUADRAT ===");
console.log("Lambat:", cekKuadratLambat([1,2,3,4,5]));
console.log("Cepat :", cekKuadratCepat([1,2,3,4,5]));

function generateArray(n) {
    return Array.from({length: n}, () => Math.floor(Math.random() * 1000));
}

const arr1 = generateArray(5000);
const arr2 = generateArray(5000);
console.log("=== BENCHMARK ===");

let t = Date.now();
IntersectionLambat(arr1, arr2);
console.log("Intersection Lambat:", Date.now() - t, "ms");

t = Date.now();
IntersectionCepat(arr1, arr2);

```

```

console.log("Intersection Lambat:", Date.now() - t, "ms");

t = Date.now();
IntersectionCepat(arr1, arr2);
console.log("Intersection Cepat :", Date.now() - t, "ms");

const arrKecil = generateArray(200);

t = Date.now();
cekKuadratLambat(arrKecil);
console.log("Kuadrat Lambat:", Date.now() - t, "ms");

t = Date.now();
cekKuadratCepat(arrKecil);
console.log("Kuadrat Cepat :", Date.now() - t, "ms");

```

```

=== INTERSECTION ===
Lambat: (2) [2, 3]
Cepat : (2) [2, 3]

=== ANAGRAM ===
[eat, tea, ate] [tan, nat] [bat]

=== KUADRAT ===
Lambat: true
Cepat : false

=== BENCHMARK ===
Intersection Lambat: 45 ms
Intersection Cepat : 1 ms
Kuadrat Lambat: 1 ms
Kuadrat Cepat : 1 ms

```

6. Input dan Output Tugas 2

```
function fn_O1(n) {
    return n + 1;
}

function fn_On(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        total += i;
    }
    return total;
}

function fn_OnLogn(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            count++;
        }
    }
    return count;
}

function fn_On2(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            count++;
        }
    }
    return count;
}

function ukur(fn, n) {
    const mulai = Date.now();
    fn(n);
    const selesai = Date.now();
    return selesai - mulai;
}

function benchmarkKhusus(ukuranData) {
    for (let n of ukuranData) {
        console.log(`===== n = ${n} =====`);

        console.log("O(1)      :", ukur(fn_O1, n), "ms");
        console.log("O(n)      :", ukur(fn_On, n), "ms");
        console.log("O(nlogn):", ukur(fn_OnLogn, n), "ms");

        console.log("O(n^2)   :", ukur(fn_On2, n), "ms");
    }
}

benchmarkKhusus([100, 500, 1000, 5000, 10000]);
```

```
=== n = 100 ===
O(1)      : 0 ms
O(n)      : 0 ms
O(nlogn): 1 ms
O(n^2)    : 0 ms
```

```
=== n = 500 ===
O(1)      : 0 ms
O(n)      : 0 ms
O(nlogn): 0 ms
O(n^2)    : 1 ms
```

```
=== n = 1000 ===
O(1)      : 0 ms
O(n)      : 0 ms
O(nlogn): 0 ms
O(n^2)    : 2 ms
```

```
=== n = 5000 ===
O(1)      : 0 ms
O(n)      : 0 ms
O(nlogn): 0 ms
O(n^2)    : 30 ms
```

```
=== n = 10000 ===
O(1)      : 0 ms
O(n)      : 0 ms
O(nlogn): 0 ms
O(n^2)    : 118 ms
```

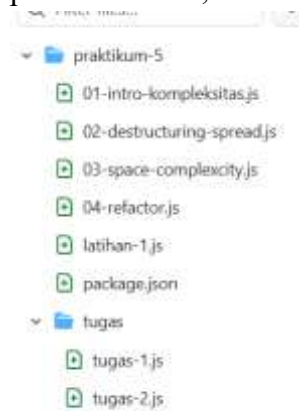
3.4 Summit ke GitHub

1. Pertama kita masuk git bash dan ketik :
2. `cd /d.`
3. Masuk ke folder yang sudah disiapkan sebelumnya `cd "/d/praktikum algoritma dan struktur data"`.
4. Masuk ke repository : `cd 2025573010104_strukturdataalgoritma.`
5. Kemudian masuk ke folder praktikum-5: `cd praktikum-5 .`
6. Kembali ke repository : `cd ..`
7. Ketik : `git add.`
8. Ketik : `git commit -m "Praktikum 5: Analisi Kompleksitas Algoritma"`
9. Ketik : `git push origin main.`

10. Tunggu beberapa saat proses summit sedang berjalan, hingga muncul seperti ini:

```
lzzah0122@0122:~$ cd /d/praktikum algoritma dan struktur data/2025573010104_strukturdatadanalgori
lzzah0122@0122:~$ git add .
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time G
it touches it
lzzah0122@0122:~$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 3/13/2021] Praktikum 5: Analisis Kompleksitas Algoritma
0 files changed, 482 insertions(+)
create mode 100644 praktikum-5/01-intro-kompleksitas.js
create mode 100644 praktikum-5/02-destructuring-spread.js
create mode 100644 praktikum-5/03-space-complexity.js
create mode 100644 praktikum-5/04-refactor.js
create mode 100644 praktikum-5/latihan-1.js
create mode 100644 praktikum-5/package.json
create mode 100644 praktikum-5/tugas/tugas-1.js
create mode 100644 praktikum-5/tugas/tugas-2.js
lzzah0122@0122:~$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 20 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.93 MiB | 1.64 MiB/s, done.
Total 12 (delta 11), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 1 local object.
to https://github.com/lzzahrizq602/2025573010104_strukturdatadanalgoritma.git
229b6bf..3711203 main -> main
```

11. Selanjutnya, buka GitHub melalui browser dan masuk ke bagian repository. Jika proses berhasil, maka tampilannya akan terlihat seperti berikut.



BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil praktikum dan pembahasan yang telah dilakukan, dapat disimpulkan bahwa algoritma memiliki peran yang sangat penting dalam menentukan kinerja suatu program. Pemahaman terhadap konsep Big O Notation memungkinkan seorang programmer untuk menganalisis efisiensi algoritma, baik dari segi waktu maupun penggunaan memori. Hasil pengujian menunjukkan bahwa algoritma dengan kompleksitas yang lebih kecil, seperti $O(n)$, memiliki performa yang jauh lebih baik dibandingkan dengan algoritma dengan kompleksitas lebih besar seperti $O(n^2)$ atau $O(n^3)$, terutama ketika digunakan pada data dalam jumlah besar.

Selain itu, penggunaan struktur data seperti Set terbukti mampu meningkatkan efisiensi dengan mempercepat proses pencarian data. Namun demikian, peningkatan kecepatan ini sering kali diikuti dengan penggunaan memori tambahan, yang menunjukkan adanya konsep time-space trade-off. Oleh karena itu, dalam merancang algoritma, diperlukan pertimbangan yang matang antara efisiensi waktu dan penggunaan memori agar sesuai dengan kebutuhan sistem yang dikembangkan.

Dengan demikian, dapat disimpulkan bahwa pemilihan algoritma yang tepat sangat berpengaruh terhadap performa program. Algoritma yang efisien tidak hanya mempercepat proses eksekusi, tetapi juga memungkinkan program untuk menangani data dalam skala besar dengan lebih optimal.

4.2 Saran

Dalam praktikum ini, praktikan sebaiknya lebih teliti dalam menulis kode agar tidak terjadi kesalahan kecil seperti penulisan variabel yang sama, typo, atau kesalahan sintaks yang bisa menyebabkan error. Selain itu, penting untuk membaca dan memahami instruksi soal terlebih dahulu sebelum mulai mengerjakan agar tidak salah arah. Praktikan juga disarankan untuk mencoba menjalankan program dengan berbagai ukuran data, baik kecil maupun besar, sehingga dapat melihat perbedaan performa dari setiap algoritma secara langsung. Dengan cara ini, praktikan akan lebih mudah memahami konsep kompleksitas yang dipelajari. Jika mengalami kesulitan, praktikan dapat mencari referensi tambahan dari internet atau berdiskusi dengan teman agar pemahaman menjadi lebih luas. Dengan latihan yang rutin dan konsisten, kemampuan dalam membuat dan menganalisis algoritma yang efisien akan semakin meningkat.